SIFT — Data-Analyst Agent on Messy Databases

Hackathon track: AI-02

Theme: Building Intelligent, Trustworthy and Sustainable Systems for the Future

Overview

SIFT is a text-to-SQL agent that lets business users ask questions in plain English against real-world relational databases. It handles the full pipeline autonomously:

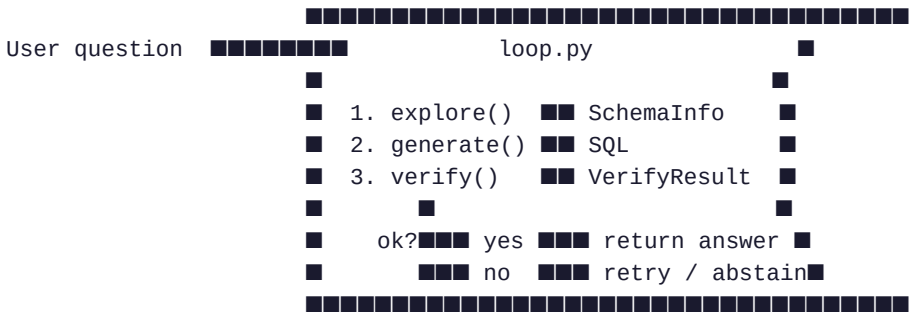
1. **Explores** the database schema, surfacing dirty values and NULL patterns.
2. **Generates** a SQL query using a local open-weight language model (≤ 8 B parameters).
3. **Verifies** the query by executing it and running sanity checks.
4. **Repairs** the query if checks fail, up to a configurable retry cap.
5. **Abstains** and says "cannot answer" when the question is unanswerable — rather than returning a plausible wrong answer.

Evaluated on the **BIRD** benchmark (Execution Accuracy).

Hackathon Constraints Met

Constraint	How we meet it
Open-weight models ≤ 8 B parameters	Qwen2.5-Coder-7B-Instruct (7.6 B, Apache 2.0)
Official benchmark, re-runnable script	BIRD dev split; `python -m src.eval.run_eval`
No personal data collected	Only SELECT queries; no data written anywhere
Every source documented	`docs/SOURCES.md`
Phase 2 extensible	Four modules with stable interfaces; config-driven

Architecture



Module responsibilities

Module	File	Purpose
explore	`src/agent/explore.py`	Read schema via PRAGMA; sample distinct values per column; detect NULLs and dirty values ("N/A", empty strings)
generate	`src/agent/generate.py`	Build a prompt (schema + evidence + question + error feedback) and call the LLM; extract SQL from response
verify	`src/agent/verify.py`	Execute SQL read-only with timeout; flag execution errors, empty results, high NULL ratios
abstain	`src/agent/abstain.py`	Decide when to say "cannot answer" (hard SQL errors after all retries)
loop	`src/agent/loop.py`	Orchestrate the above; expose CLI entry point
llm/client	`src/llm/client.py`	REST wrapper over Ollama (default) or vLLM

Handling Messy Databases

Real databases have several failure modes SIFT is designed to handle:

Problem	SIFT's response
Columns with "N/A" or empty strings	`explore` surfaces them as sample values; prompt instructs the model to filter them
Mixed date formats	Prompt instructs use of `STRFTIME`/string slicing, not date arithmetic
NULL-heavy columns	`explore` flags `[has NULLs]`; `verify` checks NULL ratio and triggers repair
Wrong table or column selected	`verify` catches empty results or errors; `generate` gets error feedback on retry
Cryptic column names	Sample values in the prompt give semantic context
Question about missing data	`abstain` returns "cannot answer" instead of guessing

---

Self-Repair Loop

```
attempt 1 → generate SQL → verify
              ↑                ■ fail
              ■■■ error feedback■
attempt 2 → generate SQL → verify
              ↑                ■ fail
              ■■■ error feedback■
attempt 3 → generate SQL → verify
```

```
                ■ still failing
            abstain / best-effort
```

The error feedback passed back to the model includes:

- The SQL error message (e.g. "no such column: salary")
- Warnings (e.g. "0 rows returned — condition may be too restrictive")

Evaluation

Primary metric — Execution Accuracy (EX)

Two queries match if they return **the same result set**. Column names and ordering do not matter.

```
EX = correct / total
```

Run on the official BIRD dev split (1,534 questions, 11 databases).

Abstention metric

We maintain a curated set of ~10–50 unanswerable questions. We report:

- **Abstain Recall** — fraction of unanswerable questions where the agent correctly abstained.

Failure categories tracked

- Wrong table selected
- Dirty value not handled
- Missing domain knowledge
- Bad SQL / syntax after retries
- Incorrect abstention (false refusal)

Quickstart

Prerequisites

- Python 3.10+
- Ollama running locally
- BIRD dev dataset (see `docs/SOURCES.md` for download instructions)

Install

```
pip install -r requirements.txt
```

Pull the model

```
ollama pull qwen2.5-coder:7b
```

Ask one question

```
python -m src.agent.loop \
  --db california_schools \
  --question "Which schools had falling enrolment last year?" \
  --verbose
```

Run full evaluation

```
python -m src.eval.run_eval --split dev --config configs/default.yaml
```

Run tests

```
pytest -q
```

Configuration

All knobs are in `configs/default.yaml`:

```
model:
  provider: ollama          # ollama | vllm
  name: qwen2.5-coder:7b
  temperature: 0.0          # deterministic output

agent:
  max_retries: 3            # repair attempts before abstaining
  query_timeout: 30         # seconds per SQL execution
  null_ratio_threshold: 0.7 # trigger repair if >70% NULLs in result
```

Change one thing at a time and record EX before and after in `docs/results.md`.

Project Layout

```
sift/
■■■ src/
■   ■■■ agent/          explore · generate · verify · abstain · loop
■   ■■■ llm/            Ollama / vLLM REST client
■   ■■■ eval/           run_eval · abstain_eval
■■■ configs/            default.yaml
■■■ data/               BIRD databases (gitignored — download separately)
■■■ docs/               SOURCES.md · results.md
■■■ tests/              37 unit tests
```

Results

See `docs/results.md` for the running log. Baseline not yet run — model and dataset downloads in progress.

Licence & Sources

See `docs/SOURCES.md` for all data and model sources with licences.

- **Qwen2.5-Coder-7B-Instruct** — Apache 2.0
- **BIRD benchmark** — CC BY 4.0 (questions); individual DB licences vary
- **Ollama** — MIT